



A CAPSTONE PROJECT – I REPORT

ON

“MayaJaal – Local-First Cyber Deception and Endpoint Defense Platform”

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY (B.Tech.)

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

MEET BAROT

ENROLLMENT NO: 2301201110

MANAV PATEL

ENROLLMENT NO: 2301201119

Under the Guidance of

MS. MITI DESAI

Drs. Kiran & Pallavi Patel Global University (KPGU)

Krishna School of Emerging Technology and Applied Research

Department of Computer Science and Engineering

VADODARA, GUJARAT

ACADEMIC YEAR: 2026–2027

CERTIFICATE

This is to certify that the Capstone Project – I Report entitled “MayaJaal – Local-First Cyber Deception and Endpoint Defense Platform” submitted by Meet Barot, Enrollment No. 2301201110, and Manav Patel, Enrollment No. 2301201119, in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, has been successfully completed during the academic year 2026–2027.

This work has been carried out under the guidance and supervision of Ms. Miti Desai and is up to our satisfaction.

Date: _____

Place: Vadodara

Ms. Miti Desai

Internal Guide

Head of the Department

Department of Computer Science and
Engineering

DECLARATION

We hereby declare that the Capstone Project – I Report entitled “MayaJaal – Local-First Cyber Deception and Endpoint Defense Platform” is a record of the original work carried out by us during the course of our Capstone Project – I.

This work has not been submitted, either in part or in full, to any other University or Institution for the award of any degree or any other academic qualification.

We further declare that all sources of information used in this report have been duly acknowledged and cited wherever necessary.

Date: _____

Place: Vadodara

Student 1 Signature: _____

Name: Meet Barot

Enrollment No: 2301201110

Student 2 Signature: _____

Name: Manav Patel

Enrollment No: 2301201119

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Ms. Miti Desai, our project guide, for her continuous guidance, valuable suggestions, constructive feedback, and encouragement throughout the development of our Capstone Project – I titled “MayaJaal – Local-First Cyber Deception and Endpoint Defense Platform.”

We are grateful to the Department of Computer Science and Engineering, Drs. Kiran & Pallavi Patel Global University (KPGU), Vadodara, and the Krishna School of Emerging Technology and Applied Research (KSET) for providing us with the academic environment, resources, and opportunities required to carry out this project.

We would also like to express our appreciation to the faculty members and staff of the department for their support and guidance during the development of the project. Their insights into software engineering practices, operating system concepts, and cybersecurity principles helped us refine both the architecture and the security posture of MayaJaal.

Finally, we extend our heartfelt thanks to our families, friends, and everyone who directly or indirectly supported us throughout this project. Their encouragement and motivation helped us overcome challenges and complete this work successfully.

ABSTRACT

Endpoint security has evolved beyond signature-based malware detection. Contemporary threats frequently involve insider data exfiltration, removable-media copying, credential probing, and ransomware-like file modification. In such cases, individual telemetry signals may appear weak or ambiguous when examined in isolation, yet become highly significant when correlated with deception artifacts and behavioral context. Academic laboratories and single-workstation environments also face a practical gap: commercial Endpoint Detection and Response (EDR) platforms are often costly, cloud-dependent, and opaque in their scoring logic, which limits their usefulness for teaching, auditing, and controlled demonstration.

MayaJaal – Local-First Cyber Deception and Endpoint Defense Platform addresses this gap by integrating decoy-based deception, multi-signal behavioral scoring, policy-driven response selection, safety gating, cryptographic containment, and operator visualization into a coherent Windows prototype. The system comprises two cooperating processes. The Guardian host collects file-system, Universal Serial Bus (USB), and process signals; maintains a sliding threat window; evaluates risk, confidence, and named correlation patterns; and executes approved containment actions. The Security Center presents live threat state to an operator through a Windows Presentation Foundation (WPF) console connected over a named-pipe inter-process communication (IPC) channel.

The implemented platform focuses on five functional pillars: (i) deception and telemetry collection through six honey-file templates and endpoint collectors; (ii) transparent threat scoring via Risk, Confidence, and Correlation engines with explicit, unit-tested formulas; (iii) policy and safety-gated response selection to reduce unsafe automation; (iv) reversible AES-256-GCM vault lockdown with Argon2id key derivation, SQLite persistence, SHA-256 hash chaining, and incident evidence packaging; and (v) operator visualization and control through Security Center. Supporting operational modes include console hosting, deterministic demo escalation, continuous simulation, and Windows Service installation scripts.

Technically, MayaJaal is developed in C# 12 on .NET 8 using Clean Architecture layering across Shared, Domain, Infrastructure, Guardian, and Security Center projects. Domain engines remain free of input/output side effects to enable rigorous unit testing. Infrastructure services provide persistence, cryptography, decoy deployment, incident management, and PDF report generation. Continuous integration on GitHub Actions validates Release builds and domain tests on Windows runners.

The project demonstrates that deception signals and weak behavioral indicators can be composed into high-confidence defensive decisions when scoring is transparent, responses

are confidence-gated, and containment is reversible and evidence-backed. The modular design also establishes a foundation for future enhancements such as deeper telemetry sensors, continuous integrity verification, richer automated testing, and professional packaging, without abandoning the local-first and explainable-security principles that define the Capstone contribution.

Keywords: *Cyber Deception, Endpoint Defense, Honey Files, Risk Scoring, Confidence Engine, Attack Correlation, AES-GCM Vault, Named-Pipe IPC, Windows Security, Clean Architecture, MayaJaal.*

List of Tables

Table 2.1: System Module Descriptions	14
Table 2.2: Software and Hardware Requirements	15
Table 3.1: System Layer Responsibilities	21
Table 3.2: Major Data Entities and Stores	26
Table 3.3: Named-Pipe IPC Commands	27
Table 4.1: Technology Stack and Tools	32
Table 4.2: Module Description	37
Table 4.3: Sample Test Results	40
Table 5.1: Evaluation of Major Modules	45

List of Figures

Figure 2.1: High-Level Use Context of MayaJaal 17

Figure 3.1: MayaJaal System Architecture 20

Figure 3.2: Threat Processing Pipeline 22

Figure 3.3: Event Store and Vault Data Layout 26

Figure 4.1: Technology Stack Overview 31

Figure 4.2: Security Center Dashboard Concept 42

Table of Contents

1	Certificate	i
2	Declaration	ii
3	Acknowledgement	iii
4	Abstract	iv
5	List of Tables	v
6	List of Figures	vi
7	CHAPTER 1: INTRODUCTION	1
1.1	BACKGROUND	1
1.2	PROBLEM STATEMENT	2
1.3	OBJECTIVES	3
1.4	SCOPE	4
1.4.1	Deception and Telemetry Collection	4
1.4.2	Threat Scoring Engines	4
1.4.3	Policy and Safety-Gated Response	5
1.4.4	Cryptographic Vault and Evidence	5
1.4.5	Security Center Operator Console	5
1.5	ORGANIZATION OF REPORT	5
8	CHAPTER 2: SYSTEM ANALYSIS	7
2.1	STUDY OF CURRENT SYSTEM	7
2.2	PROBLEMS AND WEAKNESSES OF CURRENT SYSTEM	8
2.3	REQUIREMENTS OF THE NEW SYSTEM	9
2.3.1	Functional Requirements	9
2.3.2	Non-Functional Requirements	10
2.4	SYSTEM FEASIBILITY	11
2.4.1	Technical Feasibility	11
2.4.2	Economic Feasibility	11
2.4.3	Operational Feasibility	12
2.5	ACTIVITIES IN PROPOSED SYSTEM	12
2.6	MAIN MODULES	13
2.7	SOFTWARE AND HARDWARE	15
2.7.1	Software Requirements	15
2.7.2	Hardware Requirements	15
2.8	PROPOSED SYSTEM OVERVIEW	16

9	CHAPTER 3: SYSTEM DESIGN	18
3.1	DESIGN METHODOLOGY	18
3.2	SYSTEM ARCHITECTURE	19
3.3	SYSTEM MODULE DESIGN	20
3.4	DECEPTION AND COLLECTOR DESIGN	21
3.5	THREAT ENGINE DESIGN	22
3.6	RISK, CONFIDENCE, AND CORRELATION DESIGN	23
3.7	POLICY AND SAFETY GATE DESIGN	24
3.8	VAULT AND CRYPTOGRAPHY DESIGN	25
3.9	DATABASE AND EVIDENCE DESIGN	25
3.10	INTERFACE DESIGN	27
3.11	SECURITY AND ACCESS CONTROL	28
3.12	SUMMARY OF SYSTEM DESIGN	29
10	CHAPTER 4: IMPLEMENTATION	30
4.1	TECHNOLOGY STACK	30
4.2	MODULE DEVELOPMENT	33
4.3	TESTING	37
11	CHAPTER 5: EVALUATION AND FUTURE WORK	43
5.1	EVALUATION OF RESULTS	43
5.2	FUTURE ENHANCEMENTS	45
5.3	SUMMARY	46
12	CHAPTER 6: CONCLUSION AND DISCUSSION	48
6.1	CONCLUSION	48
13	Appendix	50
14	Reference	52

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND

Information systems in educational institutions, research laboratories, and small organizations increasingly depend on Windows workstations as the primary environment for creating, storing, and exchanging sensitive documents. These endpoints commonly contain financial worksheets, credential notes, strategic plans, personal records, and application secrets. As a result, the workstation has become a high-value target for both external malware operators and trusted insiders who already possess legitimate local access.

Historically, endpoint defense emphasized antivirus signatures and perimeter controls. While these measures remain necessary, they are insufficient against many modern abuse patterns. An insider may insert a USB drive and copy large portions of a Documents folder without dropping a malicious binary. A credential-harvesting tool may touch only a few attractive files. Ransomware-like activity may appear first as rapid rename and modify bursts rather than as a previously known executable. In each case, waiting for a definitive malware label can allow irreversible data loss or disclosure.

Commercial EDR products attempt to close this gap through broad telemetry and analytics, often backed by cloud services. Such platforms are powerful in enterprise settings, yet they introduce constraints for Capstone and laboratory contexts: licensing cost, dependency on external connectivity, and limited visibility into the exact mathematical basis of alerts. For students of Computer Science and Engineering, an opaque cloud score is pedagogically weaker than a system whose risk and confidence calculations can be inspected, tested, and explained during a viva voce examination.

Deception technology offers a complementary principle. Instead of only watching real assets, the defender plants attractive but synthetic assets—honey files or decoys—where unauthorized interest becomes an immediate high-value signal. A decoy that resembles a password store, payroll export, or strategic memorandum is unlikely to be opened during ordinary work. When such a file is accessed in combination with USB activity or mass copying, the defensive narrative becomes substantially stronger than any one signal alone.

MayaJaal, a name suggesting an “illusion” or “web of deception,” is conceived as a local-first Windows cyber-defense prototype that unifies these ideas. It deploys multiple decoy templates, collects endpoint telemetry, scores a sliding threat window with explicit engines, correlates named attack patterns, and can contain impact through reversible cryptographic vault lockdown while retaining durable evidence. A dedicated Security Center console makes the resulting threat state visible to an operator without requiring a

cloud control plane.

From an academic perspective, the project integrates several core areas of the undergraduate curriculum: software architecture, operating-system services, cryptography, database persistence, human–computer interaction for security operators, and threat-informed design. The Capstone therefore is not merely an assembly of application programming interfaces; it is a complete detect–decide–contain narrative implemented as working software.

1.2 PROBLEM STATEMENT

The central problem addressed by MayaJaal is decision-making under uncertainty on a local Windows endpoint. Defenders must determine when ordinary-looking activity should escalate into alert or containment. A USB insertion may be legitimate. A burst of file operations may be a student rearranging coursework. A process start may be a new development tool. Yet combinations of these events with honey-file access can indicate active exfiltration or destructive intent.

Current practice in many small environments leaves this decision either to human vigilance—which does not scale—or to commercial agents whose internals cannot be audited in coursework. Specific weaknesses include:

1. Fragmentation of local visibility across antivirus, device logs, and ad-hoc scripts, with little correlation among signals.
2. Limited use of deception artifacts as first-class sensors inside a scoring pipeline.
3. Absence of explicit confidence thresholds before high-impact automated actions.
4. Dependence on cloud analytics that may be unavailable in offline laboratories.
5. Weak local evidence integrity, making later review of incidents difficult.
6. Insufficient pedagogical transparency for students who must justify why a system escalated.

Accordingly, there is a need for an integrated local platform that can deploy and monitor honey files; collect file, USB, and process telemetry into a common event model; compute risk with time decay and honey-aware weighting; estimate multi-factor confidence; recognize named multi-signal patterns; select responses through policy; apply a safety gate before disruptive actions; persist events with integrity-friendly storage; and present the outcome through an operator console. MayaJaal is designed expressly to satisfy this combined requirement set within Capstone scope.

1.3 OBJECTIVES

The primary objective of this Capstone Project is to design, implement, and evaluate a local-first cyber deception and endpoint defense platform that demonstrates transparent scoring and controlled containment on Windows. The specific objectives are as follows:

1. To develop a Guardian host capable of collecting endpoint security signals and executing a continuous threat-evaluation pipeline.
2. To implement honey-file deception with multiple decoy templates representing sensitive document categories.
3. To design and implement Risk, Confidence, and Correlation engines with explicit formulas and automated unit tests.
4. To implement PolicyEngine and SafetyGate components that map threat state to response actions under confidence constraints.
5. To provide AES-256-GCM vault lockdown with Argon2id-derived wrapping keys for reversible containment.
6. To persist security events in SQLite using sequence numbers and SHA-256 hash chaining, and to generate incident evidence artifacts.
7. To develop a Security Center WPF console that visualizes threat metrics, events, incidents, vaults, and policies over named-pipe IPC.
8. To support demo, simulate, console, and Windows Service operational modes suitable for teaching, verification, and local deployment.
9. To document architecture, algorithms, limitations, and evaluation results in a professional Capstone report.

1.4 SCOPE

The scope of Capstone Project – I covers the complete design and implementation of MayaJaal as a modular Windows prototype. The following subsections define the major functional areas included in the work.

1.4.1 Deception and Telemetry Collection

This area includes deployment of six honey decoy templates under Documents\MayaJaal-Decoys; FileSystemWatcher-based monitoring of Documents and decoy directories; synthesis of mass-activity events when operation volume exceeds configured thresholds; USB insert/remove collection; and process-start collection for suspicious process names. A process allowlist reduces noise from trusted system processes, while honey-related events always bypass suppression so deception signals are never discarded as benign noise.

1.4.2 Threat Scoring Engines

This area covers the RiskEngine, ConfidenceEngine, and CorrelationEngine operating over a sliding in-memory threat window of approximately fifteen minutes and at most five hundred events. Risk is computed as a time-decayed weighted sum with a soft cap. Confidence combines signal strength, correlation quality, context consistency, and historical accuracy. Correlation detects named set-membership patterns such as USB_Honey and USB_Process_MassCopy. These engines are implemented as pure Domain components to support deterministic testing.

1.4.3 Policy and Safety-Gated Response

This area includes mapping threat level and confidence to response actions such as monitor, alert, contain, and emergency lockdown, and applying SafetyGate checks that may approve or block high-impact actions. Incident creation, status updates, resolve operations, and rollback-oriented command paths are included so automated defense remains reviewable and, where appropriate, reversible.

1.4.4 Cryptographic Vault and Evidence

This area covers vault create, unlock, and lock flows; AES-GCM item encryption; Argon2id key derivation; DPAPI-protected bootstrap secrets for local demo convenience; SQLite event persistence with hash chaining; incident evidence JSON; and QuestPDF incident report generation. The design intentionally prefers reversible containment over destructive wiping of user data.

1.4.5 Security Center Operator Console

This area covers the WPF MVVM Security Center that polls Guardian approximately every two seconds for status, events, incidents, vaults, and policies; displays an offline banner when IPC is unavailable; and supports selected operator actions such as manual vault lock where exposed by the IPC contract. The console is treated as a thin client of Shared contracts rather than a direct consumer of Domain or Infrastructure types.

Explicitly outside Capstone scope are kernel minifilters, full packet capture, multi-tenant cloud fleet management, claims of commercial EDR replacement, and formal Common Criteria or FIPS certification. These boundaries are stated to keep evaluation criteria honest and achievable.

1.5 ORGANIZATION OF REPORT

The remainder of this report is organized to move from problem framing to analysis, design, implementation, evaluation, and conclusion.

- Chapter 2 presents system analysis, including study of current systems, weaknesses, requirements, feasibility, activities, modules, and software/hardware needs.

- Chapter 3 describes system design, architecture, collectors, engines, vault and evidence design, interfaces, and security controls.
- Chapter 4 details implementation, technology stack, module development, testing, demo narrative, and implementation challenges.
- Chapter 5 evaluates results, discusses strengths and limitations, and outlines future enhancements.
- Chapter 6 concludes the work and discusses design trade-offs and academic significance.
- The Appendix provides normative tables, worked examples, catalogs, and reproduction guidance. References list cited sources.

CHAPTER 2: SYSTEM ANALYSIS

2.1 STUDY OF CURRENT SYSTEM

Workstation security in contemporary practice is typically assembled from several independent layers. Antivirus software scans files and processes against known indicators. Operating-system protections such as User Account Control, Windows Defender features, and removable-storage policies provide baseline hardening. In larger organizations, EDR agents stream telemetry to a centralized analytics plane. Separately, administrators may inspect Event Viewer logs, enable auditing policies, or deploy Data Loss Prevention (DLP) rules that classify content leaving the endpoint.

In academic laboratories, the situation is often simpler and more fragmented. Students and faculty may rely on built-in antivirus, occasional manual checks, and course-specific tools. Network honeypots, where present, observe external scanning rather than local document theft. Sample programs used in operating-system or security courses demonstrate individual APIs—for example file watchers or encryption routines—but rarely compose them into an end-to-end defensive workflow with incidents, evidence, and an operator console.

A careful study of this landscape reveals both capabilities and gaps. Antivirus remains effective against many commodity threats. EDR is effective in enterprises with budget and operations staff. DLP helps when content classification is mature. None of these, however, simultaneously satisfy the Capstone needs of local operation, deception-centered correlation, explainable scoring, reversible containment, and classroom demonstrability. MayaJaal is therefore positioned not as a replacement for all existing tools, but as an integrated local prototype that occupies the pedagogical and architectural space between toy samples and opaque commercial suites.

Figure 2.1: High-Level Use Context of MayaJaal

Figure 2.1 (conceptual) places the workstation user, potential insider or malware process, removable media, operating-system file APIs, optional antivirus/EDR, Guardian, Security Center, and the human operator in one context. MayaJaal inserts decoys and a local scoring host into this environment and presents results through the operator console.

2.2 PROBLEMS AND WEAKNESSES OF CURRENT SYSTEM

Analysis of existing approaches identifies the following principal weaknesses relevant to the Capstone problem:

1. Fragmented Local Visibility

File activity, USB events, and process starts are frequently observed by different tools, if they are observed at all. Without a shared event model, correlation becomes an informal mental exercise performed by an administrator after damage has occurred.

2. Weak Integration of Deception

Honey files, when used, may generate isolated alerts. They are rarely combined systematically with mass-copy detection and process context inside one local pipeline that can also act on the result.

3. Opaque Scoring and Limited Auditability

Commercial engines seldom expose normative formulas suitable for academic audit, unit testing, or classroom explanation. Students cannot easily verify why a score crossed a threshold.

4. Unsafe or Absent Automation

Systems that automate containment without confidence gates risk disrupting legitimate work. Systems that only alert may leave sensitive data exposed for too long. A balanced design needs both decisive action and safety checks.

5. Fragile Evidence Trails

Local logs can be incomplete, rotated away, or altered. Without append-friendly integrity mechanisms and incident packaging, post-incident review is weak—especially in teaching environments where reproducibility matters.

6. Cloud Dependence for Teaching Labs

Many modern platforms assume continuous cloud connectivity. Offline laboratories, controlled networks, and viva demonstrations benefit from a local-first architecture that can run without external services.

2.3 REQUIREMENTS OF THE NEW SYSTEM

2.3.1 Functional Requirements

The proposed system shall satisfy the following functional requirements:

1. Deploy a configurable set of honey decoy files representing sensitive document categories under the user Documents tree.
2. Monitor file-system activity in Documents and decoy directories and map relevant operations to typed security events, distinguishing honey events from ordinary file events.
3. Detect burst or mass file activity within a short time window and emit a dedicated mass-activity event with de-duplication.

4. Collect USB insertion and removal events and represent them in the common security-event model.
5. Collect suspicious process-start events while allowing trusted processes to be skipped, except that honey events must never be suppressed by allowlisting.
6. Maintain a sliding threat window and compute risk, confidence, and correlation pattern matches after each relevant event.
7. Select response actions through a policy engine and apply safety-gate checks before high-impact execution.
8. Create and update incidents, execute approved containment including vault lockdown, and support resolve and rollback-oriented command paths.
9. Persist events to SQLite with sequence numbers and hash-chain fields, and write incident evidence artifacts suitable for later review.
10. Expose status and control operations to Security Center via length-prefixed JSON over a named pipe.
11. Provide a WPF dashboard for threat metrics, timelines/events, incidents, vaults, and policies, including clear offline indication.
12. Support console, simulate, demo, and Windows Service hosting modes with scripted setup and installation aids.

2.3.2 Non-Functional Requirements

In addition to functional behavior, the system must meet quality attributes appropriate for a Capstone-grade security prototype:

- Reliability: Guardian should continue collecting and scoring under ordinary desktop workloads; Security Center should degrade gracefully when Guardian is offline.
- Performance: Scoring and two-second UI polling must remain responsive on typical student or developer laptops; the threat window must remain bounded.
- Security: Vault secrets must use Argon2id and AES-GCM; pipe access must be restricted by ACLs; ProgramData permissions should be hardenable by script; secrets must not appear in logs.
- Maintainability: Clean Architecture separation must keep Domain engines free of I/O; Shared contracts must isolate IPC shapes from UI internals.
- Testability: Domain engines must be unit-testable without UI or Windows Service dependencies.
- Usability: Operators must see threat level, risk, confidence, active pattern, and offline state without navigating opaque menus.
- Auditability: Formulas, enums, and schemas must be documented so reviewers can reproduce scoring decisions.

- Portability within Windows: The solution must target Windows 10/11 with .NET 8.

2.4 SYSTEM FEASIBILITY

2.4.1 Technical Feasibility

The project is technically feasible using mainstream Microsoft and open-source components. .NET 8 provides Windows Service hosting, WPF, named pipes, and asynchronous programming suitable for collectors and IPC. FileSystemWatcher, drive polling, and process enumeration are available through supported APIs. AES-GCM is available in the platform cryptography stack; Argon2id is available through a maintained NuGet package. SQLite via Microsoft.Data.Sqlite offers zero-administration local persistence. QuestPDF can generate local incident reports. xUnit supports automated testing of pure Domain logic. Collectively, these building blocks are mature enough for a Capstone timeline while still leaving meaningful engineering challenges in composition, safety, and operator experience.

2.4.2 Economic Feasibility

Economic feasibility is high for an academic setting. The .NET SDK, xUnit, Serilog, Microsoft.Data.Sqlite, CommunityToolkit.Mvvm, and QuestPDF Community licensing for eligible users avoid mandatory cloud subscriptions for core operation. Development hardware is a standard Windows PC already available to students. No specialized appliances are required for the baseline demo, although an optional USB drive improves live removable-media demonstrations. Maintenance cost is primarily engineering time rather than recurring license fees.

2.4.3 Operational Feasibility

Operationally, MayaJaal can be exercised in console mode during development, in simulate or demo modes during presentations, and in service-install mode for a more production-like local deployment. Operators interact through Security Center without needing to understand engine internals. Because the platform is local-first, demonstrations can proceed offline. Operational caution remains necessary: decoys must stay synthetic, and automated lockdown must be demonstrated carefully so legitimate users are not disrupted. SafetyGate and confidence thresholds exist specifically to support this operational concern.

2.5 ACTIVITIES IN PROPOSED SYSTEM

The proposed system supports a complete activity chain from setup through incident review:

- Environment setup and build using PowerShell setup scripts and the .NET SDK.
- Guardian startup in console or Windows Service form, with optional simulation or deterministic demo scenarios.

- Automatic or bootstrap vault provisioning so containment capacity exists before an incident.
- Honey decoy deployment and continuous file-system monitoring.
- Ingestion of USB and process signals into the common event model.
- Continuous risk, confidence, and correlation evaluation over the threat window.
- Incident opening, containment execution, evidence packaging, and PDF reporting.
- Operator observation and selected manual lock, resolve, or rollback actions via Security Center.
- Unit-test verification of Domain engines and CI build validation on Windows runners.

2.6 MAIN MODULES

MayaJal is decomposed into modules that mirror Clean Architecture boundaries. Table 2.1 summarizes module responsibilities.

Table 2.1: System Module Descriptions

Module	Description
Shared Models and IPC	Defines SecurityEvent, Incident, ThreatState, vault/asset models, and GuardianClient / IPC message contracts used by both processes.
Domain Engines	Pure Risk, Confidence, Correlation, Policy, and SafetyGate logic without file, network, or UI side effects.
Infrastructure Services	SQLite EventStore, cryptography and key derivation, VaultService, HoneyFileService, IncidentService, and ReportGenerator.
Guardian Host	Collectors, ThreatEngine orchestration, named-pipe IpcServer, demo/simulate modes, bootstrap, and service hosting.
Security Center	WPF MVVM operator console with live polling, offline indication, and selected control actions.
Tests and Scripts	xUnit Domain tests; setup, demo, install, uninstall, and ACL hardening scripts; CI workflow.

2.7 SOFTWARE AND HARDWARE

2.7.1 Software Requirements

- Windows 10 or Windows 11 operating system
- .NET 8 SDK and runtime, including Windows desktop workload for WPF
- Visual Studio 2022 or a C#-capable editor such as VS Code / Cursor (recommended)
- PowerShell 5 or later for setup and service scripts
- Git for source control
- Optional: GitHub Actions access for CI on windows-latest runners

2.7.2 Hardware Requirements

Table 2.2: Hardware Requirements

Component	Minimum Recommendation
Processor	64-bit dual-core CPU or better
Memory	8 GB RAM (16 GB preferred when IDE and both processes run together)
Storage	At least 1 GB free for build outputs and ProgramData artifacts
Display	1366×768 or higher for comfortable Security Center use
USB	Optional removable drive for live USB demonstration scenarios

2.8 PROPOSED SYSTEM OVERVIEW

The proposed MayaJaal system consists of a Guardian process and a Security Center process collaborating over a local named pipe. Guardian is the stateful defensive host: it collects signals, stores events, scores threat state, opens incidents, and serves IPC. Security Center is the operator face of the system: it polls status and presents risk, confidence, events, incidents, vaults, and policies.

At runtime, collectors emit SecurityEvent objects into ThreatEngine. The engine updates the sliding window, may synthesize ransomware-behavior heuristics, computes risk and confidence, evaluates correlation patterns, requests a ResponseAction from PolicyEngine, and consults SafetyGate before executing containment. Approved actions may lock vaults and write evidence under %ProgramData%\MayaJaal. The operator observes outcomes in Security Center within a short polling interval. This overview establishes the analytical foundation for the detailed design in Chapter 3 and the implementation discussion in Chapter 4.

2.9 ASSUMPTIONS AND CONSTRAINTS

- The endpoint runs Windows 10/11 with a standard user profile containing a Documents folder.

- For demos, Guardian and Security Center typically run under the same interactive user context.
- Modeled adversaries are primarily user-mode actors or insiders rather than kernel-level attackers.
- Network exfiltration beyond local USB and file activity is not the primary sensing focus.
- Decoy content is synthetic and must not contain real secrets.
- A privileged local administrator can still interfere with user-mode agents; MayaJaal does not claim kernel integrity.

CHAPTER 3: SYSTEM DESIGN

3.1 DESIGN METHODOLOGY

MayaJal follows an iterative, modular methodology inspired by Clean Architecture. The essential rule is that Domain scoring and policy logic must not depend on files, pipes, databases, or user-interface frameworks. Infrastructure adapts technical capabilities to Domain needs. Guardian hosts the runtime composition. Security Center consumes only Shared contracts. This methodology enables unit testing of formulas, replacement of storage or crypto libraries, and independent evolution of the operator console.

Development proceeded in deliberate increments: first the security-event model and engine formulas; then Guardian collectors and ThreatEngine orchestration; then persistence, vault, and incident services; then Security Center and operational scripts. Demo and simulate modes were introduced early so scoring behavior could be demonstrated deterministically during development and evaluation.

3.2 SYSTEM ARCHITECTURE

The logical architecture comprises five primary projects plus tests. Table 3.1 summarizes layer responsibilities.

Table 3.1: System Layer Responsibilities

Layer / Project	Responsibility
Presentation — Security Center	WPF views/viewmodels; operator actions; IPC client only
Host — Guardian	Collectors, ThreatEngine, IpcServer, bootstrap, service hosting
Domain — MayaJal.Domain	Pure threat math and policy/safety decisions
Infrastructure	Persistence, cryptography, decoys, incidents, reporting
Shared — MayaJal.Shared	Cross-process contracts, models, GuardianClient
Tests — MayaJal.Tests	xUnit verification of Domain engines

Figure 3.1: MayaJal System Architecture

Runtime data flows from collectors into ThreatEngine, then to EventStore and Incident/Vault services. IpcServer exposes a length-prefixed UTF-8 JSON protocol on the named pipe MayaJal.Guardian. Security Center's GuardianStatusService polls this pipe approximately every two seconds. The architecture deliberately prevents the UI from calling Domain or Infrastructure types directly, preserving a clear trust and dependency boundary.

3.3 SYSTEM MODULE DESIGN

Modules interact through interfaces and message contracts. Collectors depend on IThreatEngine. ThreatEngine depends on Domain engines and Infrastructure services registered by dependency injection. Security Center depends only on Shared IPC types. This reduces accidental complexity and clarifies where validation, persistence, and presentation concerns belong.

Dependency injection is centralized so that engine implementations can be substituted in tests and so that Guardian startup remains declarative. Application-layer helpers forward registration where needed, while Guardian remains the composition root for host-specific services such as collectors and the IPC server.

3.4 DECEPTION AND COLLECTOR DESIGN

HoneyFileService deploys six templates—Financial, Credential, Strategy, HR, API, and Personal—under the user's Documents\MayaJaal-Decoys directory. Templates are synthetic and must never embed real secrets. Their purpose is attractiveness and detectability, not authenticity of confidential content.

EventCollector uses FileSystemWatcher callbacks on Documents and decoy directories, mapping operations to FILE_* or HONEY_* event types. Burst detection synthesizes MASS_FILE_ACTIVITY when operation volume exceeds a threshold of approximately twenty-five operations in ten seconds, with short de-duplication to avoid event storms. UsbCollector periodically polls removable drives and emits USB_INSERT / USB_REMOVE. ProcessCollector polls running processes and emits PROCESS_START for suspicious names, subject to ProcessAllowlist filtering. Honey-related events are never suppressed by the allowlist.

3.5 THREAT ENGINE DESIGN

Figure 3.2: Threat Processing Pipeline

ThreatEngine is the orchestration heart of Guardian. For each incoming SecurityEvent it typically: (1) optionally drops allowlisted noise; (2) persists via EventStore with hash chaining; (3) updates the sliding window; (4) may emit ransomware-behavior heuristic events; (5) computes risk; (6) computes confidence; (7) evaluates correlation patterns and may boost risk; (8) asks PolicyEngine for a ResponseAction; (9) asks SafetyGate for approval when the action is non-trivial; and (10) creates or updates incidents and executes approved responses. Startup may replay recent SQLite events into the window to reduce cold-start blindness after restart.

The in-memory window is bounded—approximately fifteen minutes and at most five hundred events—so memory use and scoring cost remain predictable. ThreatState exposed over IPC includes current risk, confidence, level, recent events, and active incident

information for the operator console.

3.6 RISK, CONFIDENCE, AND CORRELATION DESIGN

RiskEngine computes a time-decayed weighted sum over recent events and soft-caps the result to the integer interval $[0, 200]$. Each event type has a minimum and maximum base contribution and a decay constant λ per minute. Honey events receive elevated weights and a type multiplier of 1.5, with a base-weight floor of 55, so decoy interaction strongly influences the score. Additional multipliers apply for suspicious processes, USB devices, and protected assets.

Threat banding maps risk and confidence into SAFE, LOW, MEDIUM, HIGH, and CRITICAL. When confidence is below approximately 0.35 and raw risk is high, effective risk used for banding is capped near 50 so uncertain stories do not present as CRITICAL. This design encodes an important security-operations principle: escalation should reflect both severity and trust in the evidence.

ConfidenceEngine combines factors approximately as $0.4 \cdot \text{SignalStrength} + 0.3 \cdot \text{CorrelationQuality} + 0.2 \cdot \text{ContextConsistency} + 0.1 \cdot \text{HistoricalAccuracy}$. CorrelationQuality rises as USB, honey, mass, and suspicious process evidence co-occur. ContextConsistency is higher when events arrive within a short time span. HistoricalAccuracy is bootstrapped to a stable prior suitable for a prototype without long production history.

CorrelationEngine evaluates named set-membership patterns, including USB_Honey, Honey_MassCopy, USB_Process_MassCopy, Insider_Credential_Exfil, and Ransomware_Like. Patterns are intentionally named for teaching and viva explanation. If an event is marked IsHoney but typed as an ordinary FILE_* operation, the engine normalizes it to HONEY_ACCESS for matching so decoy touches are not lost.

3.7 POLICY AND SAFETY GATE DESIGN

PolicyEngine maps threat level and confidence to ResponseAction values such as NONE/MONITOR, ALERT, CONTAIN, and EMERGENCY_LOCKDOWN. For example, CRITICAL combined with confidence at or above 0.85 is associated with emergency lockdown. Lower bands produce milder actions. The policy layer answers “what should we do if we trust the story?”

SafetyGate is the last defensive checkpoint before high-impact execution. It may block lockdown when confidence is insufficient and associates rollback plans with approved actions. Separating policy desire from safety approval prevents a single component from both proposing and irrevocably executing disruptive behavior. Threshold values across ConfidenceEngine, PolicyEngine, and SafetyGate are related but not always identical; auditors should read Domain source as the authority during review.

3.8 VAULT AND CRYPTOGRAPHY DESIGN

VaultService manages password-wrapped vaults under ProgramData\MayaJaal\Vaults. A password and salt derive a wrapping key via Argon2id with design defaults such as memory cost 65536, iterations 3, and parallelism 1. A random 32-byte master key is AES-GCM-wrapped into vault metadata. Item payloads are stored as encrypted blobs with a 12-byte IV size consistent with AES-GCM practice.

DPAPI protects a default vault bootstrap secret so Guardian can auto-provision and unlock a containment vault on local machine scope for demos. LockAll encrypts or locks unlocked vaults during emergency response, providing reversible containment rather than destructive wiping. This choice aligns with Capstone ethics: demonstrate decisive response without permanently destroying student or operator data.

3.9 DATABASE AND EVIDENCE DESIGN

Figure 3.3: Event Store and Vault Data Layout

EventStore uses Microsoft.Data.Sqlite with an events table containing identifiers, sequence, timestamp, event type, source, severity, honey and protected flags, risk contribution, path, full JSON payload, and hash-chain fields. Each insert computes a SHA-256 integrity hash over a compact canonical field set chained to the previous hash or GENESIS.

Table 3.2: Major Data Entities and Stores

Entity / Store	Purpose
SecurityEvent	Normalized telemetry unit in memory and SQLite
ThreatState	Aggregated risk/confidence/level/pattern for UI
Incident	Opened case with status, actions, and evidence links
Vault / VaultItem	Encrypted containment container and items
ProtectedAsset	Assets marked for restriction/protection workflows
Evidence pack	JSON and PDF artifacts under ProgramData\Evidence

Incidents are numbered INC-{yyyyMMdd}-{counter:D4}. ReportGenerator can render incident PDFs using QuestPDF for operator and archival use. Evidence is attached at containment time so later resolution does not erase the technical trail required for evaluation and after-action review.

3.10 INTERFACE DESIGN

3.10.1 Named-Pipe IPC Interface

IPC frames are length-prefixed UTF-8 JSON: a 32-bit little-endian length followed by the payload. This framing avoids ambiguity about message boundaries on a byte stream. Table 3.3 lists principal commands.

Table 3.3: Named-Pipe IPC Commands

Command	Purpose
PING	Liveness check
GET_STATUS	Threat metrics and Guardian health
GET_EVENTS	Recent security events
GET_INCIDENT / GET_INCIDENTS	Active or historical incidents
GET_ASSETS	Protected asset inventory
GET_VAULTS	Vault inventory and state
GET_POLICIES	Policy view model for UI
LOCK_VAULT	Manual containment lock
RESOLVE_INCIDENT	Mark incident resolved
EXECUTE_ROLLBACK	Execute associated rollback plan

3.10.2 Security Center Interfaces

Security Center provides a dashboard with threat level, risk score, confidence, correlation pattern, summary indicators, and an event timeline. Navigation pages cover incidents, vaults, and policies. An offline banner appears when Guardian cannot be reached, ensuring that stale metrics are never mistaken for an all-clear condition. Visual design prioritizes operator clarity over decorative complexity.

3.10.3 Operational CLI Interfaces

Guardian supports --console, --simulate, and --demo flags. Demo mode runs a scripted escalation path intended to exercise CRITICAL lockdown under expected scoring. Simulate mode keeps synthetic telemetry alive for interactive UI walkthroughs. PowerShell scripts wrap setup, demo launch, service install/uninstall, and ACL hardening.

3.11 SECURITY AND ACCESS CONTROL

MayaJal's trust model is local Windows identity rather than application usernames and passwords. SecurePipeFactory configures named-pipe ACLs for LocalSystem, Administrators, and the current user SID, with an additional connected-client authorization check. Vault passwords never persist in plaintext. Wrapping keys are derived with

Argon2id; item encryption uses AES-256-GCM. Bootstrap secrets can be protected with DPAPI. `harden-acls.ps1` can strip inheritance on `ProgramData\MayaJaal` and apply least-privilege patterns for evidence directories.

Residual risks remain, as with any local user-mode agent: a privileged attacker on the same machine can interfere with services and files. MayaJaal does not claim kernel-level tamper resistance. Its security value is strongest as a transparent correlation and reversible-containment layer with auditable math and durable evidence.

3.12 INCIDENT LIFECYCLE DESIGN

Incidents transition through statuses such as open, active containment, and resolved depending on response execution and operator commands. Evidence is attached at containment time so later resolution does not erase the technical trail. Rollback plans associated with SafetyGate-approved actions acknowledge that automated defense should be reversible when safe to undo. Incident numbering uses a date-based scheme for easy human reference during demos and viva examinations.

3.13 OPERATOR EXPERIENCE DESIGN PRINCIPLES

Security Center design favors immediate comprehension: threat level and confidence should be visible without navigating deep menus; offline state must never look like “all clear”; event timelines should help narrate why a pattern fired. These principles mirror real SOC usability concerns even though the product scale is intentionally small. The console is therefore designed as an operator instrument rather than a marketing dashboard.

3.14 SUMMARY OF SYSTEM DESIGN

The design separates collection, pure scoring, persistence and cryptography, host orchestration, and operator UI. Deception provides high-value signals; engines compose weak signals into confidence; policy and safety gate responses; vaults and evidence make containment and review concrete. Chapter 4 describes how this design is realized in code and verified through testing and demonstration.

CHAPTER 4: IMPLEMENTATION

4.1 TECHNOLOGY STACK

MayaJaal is implemented as a multi-project .NET 8 solution using C# 12. The stack was selected for Windows fidelity, strong typing, asynchronous I/O, and mature libraries for cryptography, persistence, and desktop UI. Figure 4.1 conceptually groups technologies by role.

Figure 4.1: Technology Stack Overview

4.1.1 Language and Runtime

C# 12 on .NET 8 provides modern language features, nullable reference types, and `async/await` for collectors and IPC. Library projects target `net8.0` where possible; Guardian and Security Center target `net8.0-windows` to access Windows-specific hosting and WPF APIs. `Directory.Build.props` centralizes product metadata such as company name, product name, and version 1.0.0.

4.1.2 Guardian Host and Backend Services

`Microsoft.Extensions.Hosting` and `Windows Services` packages host Guardian. Dependency injection registers Domain engines and Infrastructure services through composition helpers, while Guardian registers `ThreatEngine`, collectors, `IpcServer`, and bootstrap components as hosted services. This approach standardizes lifetime management and graceful shutdown.

4.1.3 Security Center and UI Toolkit

WPF with `CommunityToolkit.Mvvm` implements the operator console using MVVM. Observable properties and commands keep view code thin. `GuardianStatusService` coordinates polling and updates viewmodels bound to XAML views, including dashboard, incidents, vaults, and policies.

4.1.4 Persistence, Logging, and Reporting

`Microsoft.Data.Sqlite` stores events locally without a separate database server. Serilog writes console and rolling file logs under `ProgramData` for diagnostics. `QuestPDF` generates incident PDF reports for evidence and archival workflows associated with containment.

4.1.5 Cryptography

AES-256-GCM protects vault items and wrapped keys. `Argon2id` through `Konscious.Security.Cryptography.Argon2` derives wrapping keys from passwords. SHA-256 supports hash chaining. `DPAPI` protects bootstrap secrets on the local machine.

These choices reflect current practical guidance for authenticated encryption and password-based key derivation.

4.1.6 Testing, CI, and Deployment Tools

xUnit validates Domain engines. GitHub Actions on windows-latest restores, builds, and tests the solution. PowerShell scripts publish binaries, create the MayaJal Guardian service via sc.exe, manage Start Menu shortcuts, uninstall cleanly, and harden directory ACLs. An installer folder is reserved for future MSI packaging.

Table 4.1: Technology Stack and Tools

Area	Technologies
Language/Runtime	.NET 8, C# 12
UI	WPF, CommunityToolkit.Mvvm
Host	Microsoft.Extensions.Hosting, Windows Services
IPC	Named pipes, length-prefixed JSON
Database	SQLite via Microsoft.Data.Sqlite
Crypto	AES-GCM, Argon2id, SHA-256, DPAPI
Logging / PDF	Serilog, QuestPDF
Tests / CI	xUnit, GitHub Actions (windows-latest)
Ops Scripts	PowerShell, sc.exe, dotnet publish

4.2 MODULE DEVELOPMENT

4.2.1 Shared Models and IPC Client

MayaJal.Shared defines SecurityEvent and related enums including EventType, EventSource, EventSeverity, ThreatLevel, IncidentStatus, and ResponseAction. Nested contexts capture user, process, file, and device details. GuardianClient implements the client side of the pipe protocol used by Security Center. Keeping these types shared prevents UI/host contract drift and documents the language spoken across process boundaries.

4.2.2 Domain Engines Module

RiskEngine, ConfidenceEngine, CorrelationEngine, PolicyEngine, and SafetyGate were implemented as deterministic components with no file or network I/O. This enabled focused unit tests for empty windows, honey elevation, confidence gating, decay behavior, combine-risk behavior, and named pattern detection. Normative constants—weights, multipliers, and thresholds—live beside the algorithms they govern.

4.2.3 Infrastructure Module

EventStore creates or opens events.db and appends chained events. VaultService and KeyManager implement create, unlock, lock, and encrypt flows. HoneyFileService writes decoy templates and maintains catalog metadata. IncidentService coordinates incident lifecycle and evidence. ReportGenerator produces PDFs. Dependency injection centralizes service registration for Guardian consumption.

4.2.4 Guardian Collectors and ThreatEngine

EventCollector, UsbCollector, and ProcessCollector run as hosted background work. ProcessAllowlist reduces noise. ThreatEngine integrates Domain and Infrastructure collaborators into the pipeline described in Chapter 3. DemoScenarioRunner provides a deterministic escalation path; simulate mode injects ongoing demo telemetry while keeping IPC available for the console. Vault bootstrap ensures containment capacity exists before incidents occur.

4.2.5 IPC Server and Pipe Security

IpcServer accepts connections on MayaJal.Guardian and dispatches commands to Guardian services. SecurePipeFactory applies ACLs and performs connected-client authorization checks. Expanded commands cover vaults, incidents, policies, resolve, and rollback in addition to status and events, enabling a richer Security Center without widening the UI's dependency surface beyond Shared contracts.

4.2.6 Security Center Module

MainWindow.xaml and related views present dashboard, incidents, vault, and policy pages. GuardianStatusService polls multiple GET_* commands and updates observable state. Offline detection drives a banner so operators know when metrics are stale. Figure 4.2 represents the dashboard concept emphasizing threat level, risk, confidence, active pattern, recent events, and incident summary.

Figure 4.2: Security Center Dashboard Concept

4.2.7 Operations and Demo Scripts

scripts\setup.ps1 restores, builds, and tests. scripts\run-demo.ps1 launches Guardian with --console --simulate and then Security Center. install-service.ps1 and uninstall-service.ps1 manage Windows Service installation under Program Files. harden-acls.ps1 tightens ProgramData permissions after deployment. These scripts make the system operable by evaluators who are not familiar with every project file.

Table 4.2: Module Description

Module	Description
Shared IPC/Models	Cross-process contracts and security domain types
Domain Engines	Risk, confidence, correlation, policy, safety
EventStore	SQLite persistence with hash chain
Vault/Crypto	Argon2id and AES-GCM containment
Honey/Collectors	Decoys and file/USB/process telemetry
ThreatEngine	End-to-end scoring and response orchestration
IpcServer	Named-pipe command surface
Security Center	Operator visualization and control
Scripts/CI	Setup, demo, service install, automated build/test

4.3 TESTING

Testing focused on correctness of scoring logic, reliability of build and demo paths, and practical verification of escalation behavior. Because Domain engines are pure, they received automated unit tests. Host, IPC, and UI paths were verified through controlled manual and demo scenarios, while CI ensured Release builds and tests remain green.

4.3.1 Unit Testing

xUnit tests cover RiskEngine (empty risk, honey elevation, decay, combine), ConfidenceEngine (gates and theory cases), and CorrelationEngine (pattern detection and relatedness). These cases form the regression net for scoring math and are executed locally through setup scripts and in CI.

4.3.2 Integration and Host Testing

Integration-style verification uses Guardian --demo and --console --simulate together with Security Center. These runs validate collector emission or injection, IPC availability, UI refresh, and incident/lockdown side effects on a developer workstation.

4.3.3 Functional Testing

Functional tests exercised honey deployment, mass-activity synthesis, USB/process signals (live or simulated), policy selection, SafetyGate behavior, vault lock, incident evidence writing, and resolve/rollback command paths exposed over IPC.

4.3.4 User Interface Testing

UI testing confirmed that dashboard metrics update, offline banners appear when Guardian is stopped, navigation across incidents/vaults/policies remains usable, and critical states are visually obvious to an operator.

4.3.5 Performance Considerations

Performance measures include bounded threat-window size, approximately two-second UI polling, collector poll intervals of a few seconds, and avoidance of unbounded UI collections. On typical student hardware, demo escalation remains interactive without noticeable UI freezing.

4.3.6 Security Testing

Security-focused checks included validating that vault secrets are not stored in plaintext, confirming pipe ACL configuration paths, reviewing allowlist bypass rules for honey events, and ensuring high-impact actions remain confidence-gated. Residual local privilege risks are documented rather than overclaimed.

Table 4.3: Sample Test Results

Test Area	Sample Case	Result
RiskEngine	Honey event elevates risk vs empty window	Pass
ConfidenceEngine	USB+honey+mass yields high confidence	Pass
CorrelationEngine	USB_Process_MassCopy pattern detected	Pass
Demo scenario	CRITICAL / lockdown path exercised	Pass
CI Release build	dotnet test on windows-latest	Pass
UI offline banner	Stop Guardian during poll loop	Pass

4.4 DEMO NARRATIVE WALKTHROUGH

A representative demo sequence is: USB insertion; suspicious process start; honey access; mass file activity; correlation selects USB_Process_MassCopy; risk approaches the soft cap; confidence crosses lockdown thresholds; SafetyGate approves; vaults lock; Security Center shows CRITICAL with incident details. Table 4.4 summarizes this path.

Table 4.4: Demo Escalation Steps

Step	Signal / Module	Expected Effect
1	USB_INSERT	Removable-media context
2	PROCESS_START	Suspicious tooling context
3	HONEY_* access	High-weight deception trigger
4	MASS_FILE_ACTIVITY	Volume / exfil behavior
5	Correlation + Policy	Pattern + response selection
6	SafetyGate + Vault	Approved lockdown + evidence

4.5 IMPLEMENTATION CHALLENGES

Several challenges arose during implementation. Coordinating asynchronous collectors with a consistent event model required careful normalization of timestamps, paths, and severity. Keeping Security Center responsive required clear separation between IPC awaits and observable property updates. Designing confidence thresholds that both demo reliably and avoid trivial false lockdowns required iterative tuning against simulate and demo scenarios.

Pipe security and Windows Service permissions also demanded attention. A pipe that is too open weakens local trust assumptions; a pipe that is too closed breaks student demos under common login contexts. The chosen ACL pattern balances LocalSystem, Administrators, and current-user access. Documenting formulas in parallel with code proved essential to prevent drift between Capstone narrative and implementation constants.

4.6 CONFIGURATION AND DIRECTORY LAYOUT

On a running system, MayaJaal uses %ProgramData%\MayaJaal for logs, data, vaults, incidents, evidence, and configuration, while decoys live under the user Documents tree to remain attractive and realistic. Service installs publish binaries under Program Files\MayaJaal. This split between mutable state and program binaries follows common Windows application practice and simplifies both ACL hardening and uninstallation.

Configuration is intentionally minimal for Capstone scope: hosting flags control console, simulate, and demo behavior; engine constants live in Domain code; UI polling intervals live in Security Center services. Future versions may externalize policy packs as JSON for operator tuning without recompilation.

4.7 CONTINUOUS INTEGRATION WORKFLOW

The GitHub Actions workflow runs on windows-latest with the .NET 8 SDK. It restores the solution, builds Release, and runs tests. This provides a baseline guarantee that pull requests do not silently break engine math or project compilation. Although UI automation is not yet in CI, the workflow still catches a large class of regressions early and documents an expectable verification path for evaluators.

CHAPTER 5: EVALUATION AND FUTURE WORK

5.1 EVALUATION OF RESULTS

MayaJaal was evaluated against the requirements defined during system analysis and the design commitments documented in Chapter 3. Evaluation considered functional completeness, scoring transparency, operator usability, local security controls, and verification evidence obtained from unit tests, Release builds, and demo runs. The objective was not to claim commercial EDR parity, but to determine whether the Capstone prototype honestly delivers a complete local detect–decide–contain narrative.

5.1.1 Functional Evaluation

Deception and collectors successfully produce typed events from honey access, file activity, USB changes, and suspicious processes. ThreatEngine orchestrates persistence, scoring, correlation, policy, safety, and incident response as an end-to-end pipeline. Security Center displays live state and supports key operator workflows, including offline indication when Guardian is unavailable. Demo mode provides a repeatable CRITICAL escalation narrative that exercises nearly every major module within a classroom-friendly time budget.

Vault lockdown and evidence packaging were verified as concrete side effects of approved high-confidence responses. Incidents receive durable identifiers and artifact directories under ProgramData, enabling later review during viva examination or after-action discussion. These outcomes confirm that MayaJaal is not limited to alert text; it produces operator-visible state changes and reviewable evidence.

5.1.2 Scoring and Test Evaluation

Domain unit tests covering RiskEngine, ConfidenceEngine, and CorrelationEngine passed in Release configuration. This supports the claim that normative formulas are executable and regression-protected rather than merely descriptive. Tests exercise empty windows, honey elevation, decay, combine-risk behavior, confidence gates, and named pattern detection. The educational value of explicit math—inspectable in source and validated by tests—is one of the project’s strongest outcomes.

5.1.3 Security and Operations Evaluation

AES-GCM vault lockdown, Argon2id wrapping, hash-chained SQLite events, pipe ACLs, and ACL hardening scripts collectively demonstrate a thoughtful local trust model. Service install scripts show a path from classroom demo to workstation service deployment, even though full MSI packaging remains future work. Operationally, console, simulate, demo, and service modes provide flexibility for development, presentation, and more persistent local use.

Table 5.1: Evaluation of Major Modules

Module	Evaluation Outcome
Honey and Collectors	Meets scope; emits actionable local telemetry
Risk / Confidence / Correlation	Meets scope; unit-tested formulas
Policy and SafetyGate	Meets scope; confidence-gated containment
Vault and Evidence	Meets scope; reversible lockdown and artifacts
Security Center	Meets scope; live IPC visualization
Service / Scripts / CI	Partially mature; MSI packaging still future work

5.2 FUTURE ENHANCEMENTS

Future work should extend MayaJal without abandoning its local-first and explainable-security principles. The following enhancements are prioritized by expected defensive value and engineering readiness.

5.2.1 Deeper Telemetry Sensors

Integration of richer ETW providers, process ancestry, and optional kernel minifilter callbacks would reduce reliance on polling and improve visibility into stealthier activity. Such sensors must be introduced carefully because they increase installation complexity and privilege requirements.

5.2.2 Continuous Integrity Verification

A background hash-chain verifier job can periodically validate EventStore integrity and alert on breaks, strengthening forensic assurance beyond append-time hashing alone.

5.2.3 Expanded Automated Testing

Integration tests for IPC framing, vault round-trips, collector adapters, and UI viewmodels would increase release confidence beyond Domain unit tests and reduce dependence on manual demo verification.

5.2.4 Packaging and Enterprise Operations

MSI or MSIX installers, signed binaries, central policy packs, and optional SIEM export connectors would help transition from prototype to managed fleet use while preserving the local-first core.

5.2.5 Explainable Analytics Complements

Optional anomaly models could complement correlation patterns if features and false-positive behavior remain explainable to operators. Opaque scores without narrative value would conflict with the Capstone's transparency goal and should be avoided.

5.2.6 Cross-Host Deception Fabric

Coordinated decoys across multiple laboratory machines could support blue-team exercises, provided privacy, consent, and administrative boundaries remain explicit.

5.2.7 Usability and Accessibility Polish

Guided onboarding, clearer incident narratives, and accessibility improvements in Security Center would broaden operator audiences in classrooms and SOC-training environments.

5.3 STRENGTHS AND LIMITATIONS

Principal strengths observed during evaluation include:

- Explainable scoring suitable for academic assessment and unit testing
- End-to-end local demonstration without cloud services
- Reversible cryptographic containment rather than destructive response
- Clean Architecture separating Domain math from I/O and UI
- Operator experience that surfaces offline and critical states clearly

Principal limitations acknowledged for Capstone scope include:

- User-mode sensors can miss kernel-level tampering
- Heuristic correlation may false-positive on unusual legitimate bulk work
- Automated UI and IPC tests are not yet comprehensive
- Fleet management and centralized policy distribution are out of scope
- Packaging is script-based rather than fully MSI-polished

5.4 REQUIREMENTS TRACEABILITY

Table 5.2 summarizes how major functional requirements map to implemented capabilities and evaluation evidence. This traceability supports Capstone assessment by connecting Chapter 2 requirements to concrete outcomes.

Table 5.2: Requirements Traceability (Selected)

Requirement Theme	Implementation Evidence	Evaluation Method
Honey deception	HoneyFileService + HONEY_* events	Demo / functional check
Multi-signal collection	File/USB/process collectors	Demo / simulate runs
Transparent scoring	Domain engines + unit tests	xUnit + code review
Safety-gated response	PolicyEngine + SafetyGate	Demo lockdown path
Vault containment	VaultService LockAll	Incident/evidence artifacts
Operator console	Security Center IPC polling	UI offline/live checks

Local operations	Scripts + service install	Setup/runbook exercise
------------------	---------------------------	------------------------

5.5 ADOPTION RISKS AND MITIGATIONS

Adoption risk in real organizations includes alert fatigue and fear of accidental lockdown. Mitigations already present are confidence thresholds, SafetyGate vetoes, and reversible vault locks. Classroom adoption risk includes environment drift across student machines; mitigations include setup scripts, demo mode, and documented ProgramData layouts. These mitigations do not eliminate risk, but they make residual risk explicit and manageable within Capstone scope.

5.6 SUMMARY

Evaluation indicates that MayaJal meets its Capstone objectives as a local-first deception-and-response prototype with transparent scoring, practical containment, and an operator console. Future enhancements are natural extensions of a working core rather than repairs of a broken foundation. Chapter 6 consolidates conclusions and discusses the broader significance of the design choices.

CHAPTER 6: CONCLUSION AND DISCUSSION

6.1 CONCLUSION

MayaJaal – Local-First Cyber Deception and Endpoint Defense Platform was developed to demonstrate how deception artifacts, behavioral scoring, policy and safety controls, cryptography, and operator visualization can be combined into a coherent Windows security prototype. The work integrates a Guardian host and a Security Center console into one explainable defensive narrative suitable for Capstone evaluation.

Guardian collects file, USB, and process signals; scores a sliding threat window using Risk, Confidence, and Correlation engines; selects responses through PolicyEngine; gates them with SafetyGate; and persists events and incidents with hash-chained SQLite storage and vault-backed containment. Security Center provides live situational awareness over named-pipe IPC and makes offline or critical states unmistakable to an operator.

From a software-engineering perspective, the project applies Clean Architecture principles, dependency injection, MVVM, automated Domain testing, and scripted operations. From a security-engineering perspective, it emphasizes explainable math, reversible containment, and explicit trust boundaries rather than opaque cloud scoring alone. The Capstone implementation does not claim to replace commercial antivirus or EDR products. Instead, it shows that a student team can design and build a meaningful local defense story: weak signals become strong when correlated with honey access; high-impact actions require confidence; and evidence should remain durable and reviewable.

The modular architecture provides a foundation for future enhancements such as deeper telemetry, continuous integrity verification, broader automated tests, professional packaging, and optional explainable analytics. Overall, MayaJaal successfully fulfills the objectives of Capstone Project – I by delivering a working, documented, and demonstrable cyber-defense platform.

6.2 DISCUSSION OF DESIGN TRADE-OFFS

Several deliberate trade-offs shaped the final system. Transparency was preferred over opaque detection power so formulas remain teachable and testable. Local-first operation was preferred over cloud analytics to support offline laboratories and clear trust boundaries. Safety-gated, reversible containment was preferred over immediate destructive response to reduce harm from false positives. User-mode collectors were preferred over kernel drivers to keep installation and demonstration practical within Capstone constraints.

These trade-offs are not free. Transparency can miss stealthy techniques that proprietary models might catch. Local-first design forgoes fleet-wide learning. Reversible lockdown is

less absolute than destructive wipe. User-mode sensors can be evaded by privileged malware. The Capstone accepts these costs in exchange for an honest, inspectable, and demonstrable system.

6.3 COMPARISON WITH RELATED APPROACHES

Relative to antivirus, MayaJaal is complementary: it correlates behavior and deception rather than primarily classifying binaries. Relative to classic EDR, it is narrower and local, but more transparent for teaching. Relative to network honeypots, it focuses on endpoint document pathways. Relative to DLP, it uses decoy touch and mass-activity correlation rather than deep content classification. These comparisons clarify that MayaJaal occupies a deception-assisted local correlation and containment niche.

6.4 ETHICAL AND OPERATIONAL CONSIDERATIONS

Deception files must remain synthetic and must not trick legitimate collaborators in harmful ways. In shared laboratory machines, operators should disclose monitoring or deception software when required by policy. Automated lockdown should be demonstrated carefully so coursework or personal files are not unexpectedly made inaccessible. Evidence packs may contain paths and event metadata and should therefore be ACL-hardened; logs must never include vault passwords or raw master keys.

6.5 MAPPING TO LEARNING OUTCOMES

The project maps to several Bachelor of Technology learning outcomes. Requirements analysis appears in Chapter 2. Architectural design and modular decomposition appear in Chapter 3. Implementation with modern tooling appears in Chapter 4. Verification through tests and demos supports quality-assurance outcomes. Documentation discipline—both this Capstone report and the internal engineering specification—supports professional communication outcomes.

Cross-cutting concepts practiced include concurrency and hosting, inter-process communication, cryptography, data persistence, UI state management, scripting and DevOps basics, and threat-informed design. Taken together, these provide a stronger integrative experience than implementing any single API sample in isolation.

6.6 LESSONS LEARNED

- Pure Domain engines dramatically improve testability and documentation quality.
- Deterministic demo and simulate modes are essential for grading and presentation reliability.
- IPC contracts should evolve carefully as UI features expand.
- Safety gates are as important as detection logic when actions affect user data.

- Clear ProgramData layouts simplify both operations and report writing.
- Writing normative formulas early prevents magic-number drift across modules.

We believe the project's greatest academic contribution is not only the running software, but the clarity with which detection and response decisions can be explained, tested, and improved—an essential habit for responsible security engineering.

Appendix

A. SOURCE CODE REPOSITORY

The MayaJaal source code is organized as a .NET solution with projects under src\ and tests\, documentation under docs\, and operational scripts under scripts\. Principal paths include MayaJaal.sln, Directory.Build.props, src\MayaJaal.Guardian, src\MayaJaal.SecurityCenter, src\MayaJaal.Domain\Engines, src\MayaJaal.Infrastructure, tests\MayaJaal.Tests, docs\MAYAJAAL-COMPREHENSIVE-REPORT.md, docs\architecture.md, and .github\workflows\build.yml.

B. DEMO AND IPC VERIFICATION

Verification during development included Guardian demo and simulate runs together with Security Center IPC polling. Command classes exercised included status, events, incidents, assets, vaults, policies, lock, resolve, rollback, and ping. Evaluators may insert screenshots of Security Center and console output in hardcopy submissions as needed.

C. SAMPLE STORAGE STRUCTURE

Location / Entity	Purpose
%ProgramData%\MayaJaal\Data\events.db	Hash-chained security events
%ProgramData%\MayaJaal\Vaults	Encrypted vault containers
%ProgramData%\MayaJaal\Incidents	Incident records
%ProgramData%\MayaJaal\Evidence	Evidence packs and reports
%ProgramData%\MayaJaal\Logs	Serilog rolling logs
%ProgramData%\MayaJaal\Config	Honey catalog, assets, bootstrap secrets
%USERPROFILE%\Documents\MayaJaal-Decoys	Honey decoy files

D. BUILD AND RUN COMMANDS

- Setup: .\scripts\setup.ps1
- Guardian console: dotnet run --project src\MayaJaal.Guardian -- --console
- Simulate: add --simulate
- Demo: add --demo
- Security Center: dotnet run --project src\MayaJaal.SecurityCenter
- Live demo wrapper: .\scripts\run-demo.ps1
- Service install (administrator): .\scripts\install-service.ps1

E. THREAT SCENARIOS

Six scenarios guided requirements and acceptance planning. Each row captures a distinct detection story rather than restating a single USB narrative.

ID	Narrative	Key Signals	Intended Response
A	USB exfil with decoy probe	USB, process, honey, mass	CONTAIN / LOCKDOWN
B	Insider harvest without USB	Honey, copy, mass	ALERT / CONTAIN
C	Ransomware-like churn	Mass + ransomware heuristic	LOCKDOWN if high conf.
D	Accidental bulk delete	Deletes/mass, no honey	Usually confidence-gated
E	Operator drill	Demo/simulate inject	Deterministic walkthrough
F	Vault containment path	Approved response	LockAll + evidence

F. RISK SIGNAL WEIGHT TABLE

Table F.1 lists minimum/maximum base contribution ranges and decay constants (λ per minute) used by RiskEngine for major EventType values.

Table F.1: Signal Weights (Min, Max, λ)

EventType	Min	Max	λ / min
FILE_ACCESS	1	5	0.05
FILE_MODIFY	5	15	0.04
FILE_COPY / FILE_MOVE	10	25	0.03
FILE_DELETE	15	30	0.02
MASS_FILE_ACTIVITY	30	50	0.01
PROCESS_START	5	20	0.04
USB_INSERT	5	15	0.03
USB_FILE_ACCESS	15	35	0.02
HONEY_ACCESS	50	80	0.005
HONEY_MODIFY	60	90	0.005
RANSOMWARE_BEHAVIOR	80	100	0.001

Context multipliers further scale contributions: honey access/modify $\times 1.5$, suspicious process $\times 1.4$, USB device $\times 1.2$, protected asset $\times 1.15$. Honey events also enforce a base-weight floor of 55. Final risk is round(total) soft-capped at 200.

G. WORKED RISK EXAMPLE

Consider a single HONEY_ACCESS event with RiskContribution=70, IsHoney=true, SensitivityLevel=9, Severity=CRITICAL, and $\Delta t \approx 0$.

- $\text{baseWeight} = \max(70, 55) = 70$
- C (confidence factor for honey) ≈ 0.9
- A (sensitivity scale) $= 9/5 = 1.8$
- X (honey type multiplier) $= 1.5$
- decay term $\approx e^0 = 1.0$
- $\text{contribution} \approx 70 \times 0.9 \times 1.8 \times 1.5 \times 1.0 = 170.1 \rightarrow \text{Value} \approx 170$

If overall confidence is high, DetermineThreatLevel maps this into CRITICAL. If confidence is very low (<0.35) while raw risk is high, banding caps effective risk near 50 so uncertain stories do not immediately look CRITICAL. Unit tests cover both branches.

H. CONFIDENCE MIX AND ACTION THRESHOLDS

ConfidenceEngine computes $\text{Clamp}(0.4 \cdot \text{SignalStrength} + 0.3 \cdot \text{CorrelationQuality} + 0.2 \cdot \text{ContextConsistency} + 0.1 \cdot \text{HistoricalAccuracy})$. CorrelationQuality climbs as USB, honey, mass, and suspicious process evidence co-occur. ContextConsistency is higher when events fall inside a short time span.

Table H.1: Confidence Thresholds by ResponseAction

ResponseAction	Confidence Threshold
NONE / MONITOR	0.0
ALERT / NOTIFY_USER	0.5
LOCK_VAULT / RESTRICT_ASSETS / PRESERVE_EVIDENCE	0.7
CONTAIN	0.8
EMERGENCY_LOCKDOWN	0.9

Note for examiners: SafetyGate and PolicyEngine use related but not identical numeric gates. Auditors should treat Domain source files as the authoritative reference during code review.

I. CORRELATION PATTERN CATALOG

Table I.1: Named Set-Membership Patterns

Pattern	Required Types	TypicalRisk
USB_Honey	USB_INSERT, HONEY_ACCESS	70
Honey_MassCopy	HONEY_ACCESS, MASS_FILE_ACTIVITY	100
USB_Process_MassCopy	USB_INSERT, PROCESS_START, MASS_FILE_ACTIVITY	130
Insider_Credential_Exfil	HONEY_ACCESS, FILE_COPY, MASS_FILE_ACTIVITY	150
Ransomware_Like	MASS_FILE_ACTIVITY, RANSOMWARE_BEHAVIOR	160

If an event is marked IsHoney but typed as an ordinary FILE_* operation, CorrelationEngine normalizes it to HONEY_ACCESS for pattern matching so decoy touches are not missed due to collector typing differences.

J. VAULT CRYPTO PARAMETERS

Parameter	Value / Notes
Item encryption	AES-256-GCM
IV size	12 bytes
KDF	Argon2id
KDF memory / iterations / parallelism	65536 / 3 / 1 (design defaults)
Master key	Random 32-byte key, wrapped by KDF key
Bootstrap secret storage	DPAPI LocalMachine file under Config
Vault location	%ProgramData%\MayaJaal\Vaults\{id}

K. SQLITE EVENTS TABLE AND ARTIFACT MAP

Column / Artifact	Role / Path
id / sequence	Identity and append order
timestamp / type / source / severity	Classification fields
is_honey / is_protected	Deception and protection flags
hash_chain_* / integrity_hash	SHA-256 chain fields
events.db	%ProgramData%\MayaJaal\Data\events.db
Evidence	%ProgramData%\MayaJaal\Evidence\{id}\...
Decoys	%USERPROFILE%\Documents\MayaJaal-Decoys\

L. STRIDE REVIEW SUMMARY

Category	Example Risk	Mitigation in MayaJaal
Spoofing	Unauthorized IPC client	Named-pipe ACLs + client check
Tampering	Edited events.db	Hash chain (verifier job future)
Repudiation	Denied actions	Sequenced events + incident artifacts
Info disclosure	Evidence/log leakage	ACL harden; no secret logging
DoS	Event flood	Window cap + allowlist
Elevation	Local admin abuse	OS-owned; not claimed solved

M. UNIT TEST MATRIX AND SOURCE CATALOG

Suite / Area	Coverage / Key Paths
RiskEngine	Empty window; honey elevation; decay; combine risks
ConfidenceEngine	Low-signal gate; USB+honey+mass high confidence
CorrelationEngine	Pattern hit/miss; relatedness checks
Domain paths	src\MayaJaal.Domain\Engines*.cs
Guardian paths	ThreatEngine, Collectors, IpcServer, DemoScenarioRunner
UI paths	SecurityCenter ViewModels + MainWindow.xaml
Tests	tests\MayaJaal.Tests\UnitTests*.cs

N. HONEY DECOY TEMPLATES

HoneyFileService deploys six synthetic templates under Documents\MayaJaal-Decoys. Templates contain no real secrets. Access and modify operations are remapped to high-sensitivity honey events so unauthorized interest becomes a strong signal.

Template Category	Defensive Intent
Financial	Attract interest in payroll, invoices, or ledgers
Credential	Attract password-store or secret-file probing
Strategy	Attract interest in plans and confidential roadmaps
HR	Attract interest in personnel or offer-letter style files
API	Attract interest in keys, tokens, or integration notes
Personal	Attract opportunistic browsing of private documents

O. SECURITYEVENT FIELD SUMMARY

Field Group	Examples
Identity	Id, Sequence, Timestamp
Classification	Type, Source, Severity
Contexts	User, Process, File, Device
Scoring aids	RiskContribution, IsHoney, IsProtected
Integrity	HashChainPrevious/Current, IntegrityHash
Extensibility	Metadata dictionary; JSON payload persistence

P. WORKED CONFIDENCE EXAMPLE

Consider a short window containing USB_INSERT, PROCESS_START, HONEY_ACCESS, and MASS_FILE_ACTIVITY. CorrelationQuality climbs the ladder to approximately 0.95 because USB, honey, mass, and process evidence co-occur. SignalStrength is high because honey and mass are high-value indicators. ContextConsistency is high if the span is under ten minutes (approximately 0.85). HistoricalAccuracy uses the bootstrap prior (approximately 0.75).

A representative mix is therefore approximately $0.4 \cdot (\text{high signal}) + 0.3 \cdot 0.95 + 0.2 \cdot 0.85 + 0.1 \cdot 0.75$, which typically exceeds 0.7 and can satisfy containment or lockdown gates depending on exact signal strength and policy thresholds. This example is consistent with unit-tested USB+honey+mass confidence behavior and with the demo escalation narrative.

Q. GLOSSARY AND ACRONYMS

Guardian: Host that collects signals, scores threats, and serves IPC.

Security Center: WPF console for live visualization and operator actions.

Honey / Decoy: Synthetic sensitive-looking file used as a high-value sensor.

Threat Window: Bounded recent-event buffer used for scoring (~15m / max 500).

SafetyGate: Final approval checkpoint before high-impact responses.

Vault: Password-wrapped AES-GCM container used for containment.

Hash Chain: SHA-256 linkage across persisted events for integrity.

- AES-GCM — Authenticated encryption mode for vault items
- Argon2id — Memory-hard password-based key derivation
- EDR — Endpoint Detection and Response
- IPC — Inter-Process Communication (named pipes here)
- MVVM — Model-View-ViewModel UI pattern
- WPF — Windows Presentation Foundation

R. REPRODUCTION CHECKLIST

1. Install .NET 8 SDK on Windows 10/11.
2. Run scripts\setup.ps1 (restore/build/test).
3. Run Guardian: dotnet run --project src\MayaJaal.Guardian -- --demo
4. Or live UI: scripts\run-demo.ps1 (simulate + Security Center).
5. Confirm threat metrics and incident/evidence under ProgramData\MayaJaal.

6. Archive test output and demo observations for evaluation.

This annex provides normative tables and worked examples that complement Chapters 1–6. It is intended for viva examination and code review, not as repeated chapter prose.

Reference

1. Microsoft. .NET 8 Documentation. Microsoft Learn. Available at: <https://learn.microsoft.com/dotnet/>
2. Microsoft. Windows Services with .NET. Microsoft Learn.
3. Microsoft. Named Pipes in .NET. Microsoft Learn.
4. Microsoft. Data Protection API (DPAPI). Microsoft Learn.
5. SQLite Development Team. SQLite Documentation. Available at: <https://www.sqlite.org/docs.html>
6. Microsoft. Microsoft.Data.Sqlite Documentation. Microsoft Learn.
7. Serilog Contributors. Serilog Documentation. Available at: <https://serilog.net/>
8. Community Toolkit. MVVM Toolkit Documentation. Microsoft Learn.
9. QuestPDF Contributors. QuestPDF Documentation. Available at: <https://www.questpdf.com/>
10. xUnit.net Contributors. xUnit.net Documentation. Available at: <https://xunit.net/>
11. IETF RFC 5116. An Interface and Algorithms for Authenticated Encryption.
12. Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. IETF RFC 9106.
13. NIST. Guide to Malware Incident Prevention and Handling for Endpoints.
14. NIST. Cybersecurity Framework. National Institute of Standards and Technology.
15. OWASP. Desktop Application Security guidance and secure coding resources.
16. MITRE. ATT&CK; Matrix for Enterprise — Collection, Exfiltration, and Impact techniques.
17. Spitzner, L. Honeypots: Tracking Hackers. Addison-Wesley.
18. Provos, N., & Holz, T. Virtual Honeypots: From Botnet Tracking to Intrusion Detection. Addison-Wesley.
19. Microsoft. FileSystemWatcher Class. .NET API Browser.
20. Microsoft. WPF Documentation. Microsoft Learn.

21. SANS Institute. Endpoint Detection and Response (EDR) educational materials.
22. CIS. CIS Controls — malware defenses, audit logs, and recovery guidance.
23. ISO/IEC 27001. Information security management systems — overview and vocabulary (contextual reference).
24. MayaJaal Engineering Documentation. Comprehensive Engineering Design & Implementation Report, Document ID MJ-ENG-REP-2026-09.
25. MayaJaal Repository. README.md and docs\architecture.md — setup and architecture overview.